

FASTAPI BACKEND SECURITY AUDIT

Claude Code master prompt + evidence-based finding report template

Inspired by Alibaba OpenCodeReview's hybrid review model:
deterministic scanning + repository-aware LLM analysis + line-level findings.

IMPORTANT: This document is an audit specification and report template. It does not contain findings for your backend because the repository/source code was not supplied in this request. Run the included prompt inside the FastAPI repository to generate evidence-backed findings.

Scope: source code, API routes, authentication/authorization, data access, dependencies, configuration, deployment, secrets, tests, and abuse paths.

1. Audit Method

The review should be performed as a hybrid security audit rather than a single LLM pass. Alibaba OpenCodeReview describes this pattern as deterministic pipelines/rules combined with an LLM agent that can inspect full files, search repository context, and emit precise findings. For a FastAPI backend, the same idea is applied in four passes:

1. Inventory and attack-surface mapping: enumerate routes, auth dependencies, roles/scopes, models, external calls, storage, background jobs, configuration, and deployment assumptions.
2. Deterministic analysis: run available Python security linters, dependency vulnerability checks, tests, typing/linting, and secret/config checks.
3. Semantic security analysis: trace request input through validation, authorization, business logic, database/files/external services, and response serialization.
4. Evidence verification and reporting: keep only defensible findings, attach file/line evidence and safe reproduction steps, propose a minimal fix, add a regression test, and re-run verification.

Primary baseline: OWASP API Security Top 10 (2023), plus FastAPI-specific authentication, dependency injection, CORS, Pydantic, SQLAlchemy/ORM, file handling, JWT/session, async/background-task, and deployment risks.

2. Master Prompt for Claude Code

Paste the following prompt into Claude Code while your terminal is at the repository root. It is deliberately strict: Claude must inspect the repository, run available checks, prove findings, and create a formal report instead of only commenting on suspicious code.

You are the principal application-security engineer performing a COMPLETE SECURITY AUDIT of this FastAPI backend repository.

GOAL

Audit the entire backend for exploitable vulnerabilities, authorization mistakes, insecure assumptions, dependency risk, unsafe configuration, data exposure, and abuse cases. Behave like a repository-level security reviewer, not a style reviewer. Use a hybrid approach similar to Alibaba OpenCodeReview: deterministic scanners/checks first, then repository-aware semantic analysis, then evidence verification, then a structured report.

OPERATING RULES

- Work only against this local repository/test environment. Do not attack production, public hosts, third-party systems, or real user accounts.
- Do not perform destructive testing, persistence, credential theft, data exfiltration, or denial-of-service. Validate safely with unit/integration tests and minimal local proof-of-concept requests.
- Do not assume a finding is real because a scanner or pattern match says so. Trace the actual code path and eliminate false positives.
- Do not hide uncertainty. Every finding must include CONFIDENCE: High / Medium / Low.
- Do not modify application code until the initial audit and evidence collection are complete. Store any test-only artifacts under security-audit/.
- Never print secret values into the report. If a secret is found, redact it and record only its location/type.
- Review the WHOLE repository, not only git diff. Exclude generated/build/vendor/cache directories.
- Continue until all reachable API routers/endpoints and security-sensitive modules have been mapped and reviewed.

PHASE 0 - REPOSITORY INVENTORY

1. Identify Python/FastAPI versions, package manager, entry point, routers, middleware, settings/configuration, ORM/database layer, migrations, schemas/Pydantic models, auth modules, storage/uploads, background tasks, queues, webhooks, websocket routes, external HTTP clients, email/SMS/payment integrations, Docker/compose, reverse-proxy/deployment config and CI.
2. Build an endpoint inventory from FastAPI route declarations and included routers. For every endpoint record:
 - method and path
 - source file/function

- authentication dependency
- required role/scope/permission
- object identifier(s) accepted
- request model/body/query/path parameters
- data stores accessed
- external systems called
- response model / sensitive fields

3. Identify trust boundaries and data classes: public, authenticated-user, staff/admin, tenant/customer, secrets/tokens, PII, files, payments, privileged operations.

4. Identify whether authentication uses JWT, cookies/sessions, API keys, OAuth2/OIDC, or custom tokens and map login, refresh, logout/revocation, password reset, email verification and account recovery flows.

PHASE 1 - DETERMINISTIC CHECKS

Use existing project tooling first. Do not replace the project's dependency manager.

Run, when applicable and available:

- test suite (pytest or project equivalent)
- lint/type checks already configured in pyproject.toml/Makefile/CI
- Bandit over first-party Python code
- pip-audit against the resolved project environment/lock/requirements
- Semgrep security rules if Semgrep is already installed/configured
- secret scanning if the repository already has a scanner configured
- container/config scans if already configured in CI

Capture commands, versions, exit status, and relevant output in security-audit/tool-results/.

If a tool is missing, do not silently install arbitrary software. Record it as 'not available' and continue with manual analysis. You may propose exact install commands in the report.

PHASE 2 - MANUAL / SEMANTIC SECURITY REVIEW

Review every security-relevant path against ALL of the following categories.

A. OWASP API SECURITY TOP 10 2023

- API1 Broken Object Level Authorization (BOLA/IDOR): every endpoint that accepts IDs, UUIDs, usernames, account IDs, tenant IDs, document IDs, file IDs, etc. Verify ownership/tenant/permission checks at the data access point, not only UI assumptions.
- API2 Broken Authentication: token validation, password handling, login brute-force resistance, reset/recovery, session invalidation, refresh tokens, expiration, issuer/audience, algorithm restrictions, token reuse and user-enumeration behavior.
- API3 Broken Object Property Level Authorization: excessive response fields, Pydantic response models, mass assignment, update schemas, admin-only fields, role/owner/tenant/status fields writable by clients.
- API4 Unrestricted Resource Consumption: unbounded pagination, upload size, body size, expensive queries, search endpoints, export/report generation, image/PDF processing, background tasks, email/SMS/API cost amplification, regex/backtracking, decompression/zip bombs.
- API5 Broken Function Level Authorization: admin/staff/internal endpoints, role checks, dependency placement, router-wide vs endpoint-specific guards, method-level permission differences.
- API6 Unrestricted Access to Sensitive Business Flows: automated account creation, password/reset/email flows, invitations, purchases, bookings, submissions, approvals, exports, bulk actions, OTP/email/SMS triggers, scraping/enumeration.
- API7 SSRF: HTTPX/requests/aiohttp calls using user-controlled URLs, callbacks, webhooks, URL previews, importers, file fetchers, redirects and DNS/IP allow/block logic.
- API8 Security Misconfiguration: CORS, debug, docs/OpenAPI exposure, exception traces, trusted hosts, proxy headers, HTTPS assumptions, cookie flags, security headers, environment defaults, permissive origins, default credentials, unnecessary methods, verbose logging.
- API9 Improper Inventory Management: forgotten routers, deprecated API versions, debug/test endpoints, duplicate paths, undocumented admin/internal endpoints, exposed health/metrics, stale code still mounted.
- API10 Unsafe Consumption of APIs: trusting third-party API responses, missing timeouts, missing TLS verification, unsafe redirects, weak webhook signature validation, unvalidated callbacks/data, secrets sent to unexpected destinations.

B. FASTAPI / PYTHON SPECIFIC CHECKS

- Verify EVERY protected route actually has the intended Depends()/Security() dependency after router composition/include_router.
- Check OAuth2 scopes/roles are granted server-side and not accepted blindly from client input/token requests.
- JWT: hardcoded/weak secrets, accepted algorithms, exp/nbf/iat/iss/aud/sub validation, key rotation assumptions, sensitive data inside tokens, refresh/access token confusion, token revocation needs.
- Passwords: modern password hashing, safe verification, no plaintext/reversible storage, reset tokens single-use/expiry, no password leakage in logs/errors.
- Cookies/sessions: Secure, HttpOnly, SameSite, CSRF protection where browser cookies authenticate state-changing requests.
- CORS: reject dangerous wildcard/origin reflection combinations, especially with credentials.
- Pydantic schemas: separate create/update/internal/response models; prevent clients setting privileged fields; use response_model or equivalent filtering for sensitive data.
- SQL/ORM: raw SQL string interpolation, unsafe text(), dynamic table/column/order clauses, tenant filters, soft-delete filters, transaction boundaries, race conditions and TOCTOU authorization.
- NoSQL/search backends if present: operator injection, raw query objects, unsafe filters/scripts.
- Command execution: subprocess/os.system/shell=True, user-controlled arguments, PATH lookup, archive/convert tooling.
- Deserialization: pickle, marshal, yaml unsafe loaders, eval/exec, dynamic imports, template injection.
- File handling: traversal, absolute paths, symlink issues, filename trust, MIME/extension mismatch, executable uploads, public ACLs, overwrite collisions, temporary-file safety, max size, archive extraction.
- HTML/templates: reflected/stored XSS, unsafe HTML rendering, Jinja autoescape bypass, markdown/HTML sanitization.
- Redirects: open redirects and unvalidated return URLs.
- External HTTP: SSRF, timeout, retry amplification, TLS verification disabled, auth/header leakage across redirects/hosts.
- WebSockets/SSE: authentication at connection time, authorization per channel/resource, origin checks where relevant, disconnect cleanup, message-size/rate limits.
- Background jobs/Celery/RQ: trust boundary of job payloads, replay/duplication, idempotency, privileged worker actions, serialization.
- Logging/observability: passwords, Authorization headers, cookies, JWTs, reset tokens, API keys, PII; log injection; stack traces exposed to users.
- Secrets/config: tracked .env files, hardcoded credentials, insecure defaults, SECRET_KEY fallbacks, test credentials used in production, debug toggles.
- Dependency injection: dependencies that return unauthenticated/default users on exceptions; swallowed auth errors; optional current-user used on privileged routes.
- Multi-tenant isolation: every tenant-owned query and mutation must constrain by authenticated tenant/user, including joins, exports, bulk endpoints and background jobs.
- Rate limiting/abuse controls around authentication, OTP, reset, expensive endpoints and paid third-party actions.
- Concurrency/races: duplicate redemption, balance/order/status transitions, invite/reset token reuse, check-then-act permission races.
- Error handling: broad except blocks that fail open, suppress auth/validation failures, expose internals or return success after a failed security action.
- OpenAPI/docs: ensure sensitive/internal endpoints and schemas are not accidentally published in production if that violates intended exposure.
- Deployment: Uvicorn/Gunicorn options, bind addresses, forwarded/proxy headers, HTTPS termination assumptions, Docker user, exposed ports, volume permissions, env injection and dev server use.

PHASE 3 - SOURCE-TO-SINK / ATTACK-PATH ANALYSIS

For every suspicious flow, trace:

external input -> FastAPI parameter/schema -> normalization/validation -> authn -> authz -> business logic -> database/files/OS/network sink -> response/logging.

Do not report a vulnerability until the reachable path and missing/incorrect control are identified.

For authorization findings, identify the exact principal, target object/function, missing check and affected endpoints.

For injection/SSRF/file findings, identify the exact tainted input and dangerous sink plus any sanitization/allowlist in between.

PHASE 4 - SAFE VERIFICATION

For each Critical/High/Medium candidate, create the smallest non-destructive local verification possible:

- Prefer pytest + FastAPI TestClient/httpx against a test database.
 - Use synthetic users/objects and fake credentials only.
 - Demonstrate expected-vs-actual authorization behavior without accessing real data.
 - For dependency vulnerabilities, confirm the installed/resolved version and advisory applicability; do not claim exploitability solely from package name.
 - For configuration findings, identify the exact setting and deployment condition required.
- If verification is not possible, state exactly why and reduce confidence appropriately.

PHASE 5 - SEVERITY AND FALSE-POSITIVE TRIAGE

Classify each finding as Critical / High / Medium / Low / Informational.

Use likelihood + impact + required privileges + data sensitivity + blast radius. Include CWE and OWASP API category where applicable.

Do not inflate severity. Do not duplicate the same root cause across many endpoints; group them and list all affected routes.

Explicitly mark scanner results that are NOT exploitable as false positives / accepted observations.

MANDATORY FINDING FORMAT

For every confirmed finding include:

- Finding ID: FAPI-SEC-001 ...
- Title
- Severity
- Confidence
- Status: Confirmed / Needs Environment Verification / Informational
- CWE
- OWASP API category
- Affected endpoint(s)
- Affected file(s) and exact line(s)
- Evidence/code path
- Preconditions / attacker capabilities
- Attack scenario (concise, realistic)
- Security impact
- Safe reproduction / regression test
- Root cause
- Recommended remediation
- Minimal patch example or diff (do not apply until report is complete)
- Verification after fix: Pass / Fail / Not tested

REQUIRED DELIVERABLES

Create a security-audit/ directory containing:

1. SECURITY_AUDIT_REPORT.md - full professional report.
2. findings.json - machine-readable findings with all fields above.
3. endpoint-security-matrix.csv - one row per route with authn/authz/role/object ownership/input/response exposure/review status.
4. tool-results/ - captured deterministic scanner/test outputs with secrets redacted.
5. SECURITY_REMEDIATION_PLAN.md - fixes grouped P0/P1/P2/P3 with dependencies and regression tests.
6. tests/security/ - non-destructive regression tests for confirmed issues when practical. Do not modify existing tests unless necessary.
7. SECURITY_AUDIT_REPORT.html and SECURITY_AUDIT_REPORT.pdf if local document-generation tooling is available. Generate the PDF from the final Markdown/HTML only after the report is complete. If no PDF renderer exists, keep the Markdown/HTML and clearly record that PDF generation was unavailable rather than fabricating one.

REPORT STRUCTURE

- Cover / audit metadata (repository, branch, commit SHA, audit date, auditor=Claude Code, scope)
- Executive summary
- Overall risk posture WITHOUT a fake numeric score

- Scope and exclusions
- Repository/architecture overview
- Authentication and authorization architecture
- Attack-surface / endpoint inventory summary
- Methodology and tools actually executed
- Findings summary table (ID, severity, confidence, title, affected area, status)
- Detailed findings
- OWASP API Top 10 coverage matrix: Reviewed / Finding(s) / No confirmed finding / Not applicable
- Dependency/security-tool results
- Positive security controls observed
- Remediation roadmap P0-P3
- Verification / regression-test results
- Residual risks and environment items that require deployment/runtime validation
- Appendix: reviewed files, reviewed endpoints, commands run, tool versions

FINAL QUALITY GATE

Before finishing:

- Confirm every FastAPI route discovered is present in endpoint-security-matrix.csv and marked Reviewed or explicitly Not Reviewed with reason.
- Re-run searches for routers/decorators/include_router to detect missed endpoints.
- Re-run tests and deterministic checks after any proposed fixes are applied only if I explicitly ask you to fix them.
- Ensure every Critical/High finding has direct code evidence and a verification test or a clearly stated verification limitation.
- Ensure the report contains no secret values.
- Ensure there are no duplicate findings.
- Separate confirmed vulnerabilities from hardening recommendations.
- End with a concise list of the top remediation priorities, but do not claim the system is 'secure' merely because no additional findings were found.

START NOW.

First inventory the repository and create security-audit/endpoint-security-matrix.csv before evaluating individual findings. Then execute the deterministic checks, perform the full semantic review, verify candidates, and generate all deliverables.

3. Minimum Coverage Matrix

The final audit should explicitly cover these domains. A report that skips a row is incomplete unless it explains why the item is not applicable.

Domain	Required review coverage
Attack surface	All routers, endpoints, versions, admin/internal routes, WebSockets, background jobs
Authentication	Login, JWT/session/API keys/OAuth, password hashing, reset/recovery, token lifecycle
Authorization	BOLA/IDOR, roles/scopes, function-level permissions, tenant isolation, bulk/export paths
Input & injection	SQL/NoSQL, command, template, deserialization, path traversal, open redirect
HTTP/network	SSRF, webhooks, external APIs, TLS validation, redirects, timeouts, header leakage
Data exposure	Response models, excessive fields, logs, traces, PII, secrets, OpenAPI/docs
Files/storage	Upload limits, MIME, executable content, traversal, symlinks, cloud ACL, archive extraction
Resource abuse	Rate limits, pagination, query cost, upload/body size, cost-amplification, expensive jobs
Business logic	Sensitive workflows, replay, idempotency, race conditions, state transitions, abuse automation
Dependencies	Known vulnerabilities, abandoned packages, pinned/resolved versions, advisory applicability
Configuration	CORS, debug, trusted hosts, cookies, proxy/HTTPS, env defaults, production exposure
Deployment/CI	Containers, least privilege, ports, secrets in CI, production server config, security checks
Testing	Security regression tests, negative authorization tests, validation tests, failed-open behavior

4. Severity Model

Severity	Definition
Critical	Likely direct compromise with severe impact, e.g. unauthenticated RCE, broad auth bypass, unrestricted sensitive-data access, production secret enabling compromise.
High	Practical exploitation causing significant unauthorized access, privilege escalation, tenant escape, sensitive-data exposure, or major business impact.
Medium	Real weakness with meaningful but constrained impact, additional prerequisites, limited blast radius, or reduced exploit reliability.
Low	Minor weakness or defense-in-depth gap with low direct impact.
Informational	Hardening, maintainability, observability, or environment-dependent issue without a confirmed exploitable path.

5. Required Finding Record

Field	What must be documented
Finding ID	Stable identifier such as FAPI-SEC-001.
Title	Specific root cause, not a vague category.
Severity	Critical / High / Medium / Low / Informational.
Confidence	High / Medium / Low based on evidence and verification.
Status	Confirmed / Needs Environment Verification / Informational.
CWE	Closest applicable CWE.
OWASP API category	API1-API10 where applicable.
Affected endpoint(s)	All impacted HTTP methods/routes or background flows.
Affected file(s) and exact lines	Precise source evidence.
Evidence / code path	Input-to-sink or principal-to-resource trace showing the missing control.
Preconditions	Authentication level, role, object knowledge, deployment condition, etc.
Attack scenario	Short realistic misuse path; no sensationalism.

Impact	Concrete confidentiality/integrity/availability/business impact.
Safe reproduction / regression test	Local non-destructive proof using synthetic data.
Root cause	Why the code permits the issue.
Recommended remediation	Smallest reliable control that fixes the root cause.
Minimal patch example/diff	Illustrative change; keep behavior-compatible where possible.
Verification after fix	Pass / Fail / Not tested with command/test reference.

6. Suggested Deterministic Commands

Claude should adapt these to the project rather than blindly running them. The authoritative dependency manager/lockfile in the repository takes precedence.

```
# Existing project checks first
pytest -q
ruff check .                # only if configured/installed
mypy .                      # only if configured/installed

# Python source security scan
bandit -r . -x .env,.venv,.git,node_modules,dist,build,__pycache__ -f json -o
security-audit/tool-results/bandit.json

# Dependency vulnerability audit (adapt to environment/lock/requirements)
pip-audit --format json --output security-audit/tool-results/pip-audit.json

# Optional only when already installed/configured
semgrep --config auto --json --output security-audit/tool-results/semgrep.json .
```

Important: scanner output is evidence to investigate, not proof of exploitability. The LLM must confirm reachability, actual configuration, sanitization and authorization before promoting a scanner item into a confirmed vulnerability.

7. Example Findings Summary Table

ID	Severity	Confidence	Finding	Affected area	Status
FAPI-SEC-001	High	High	[Example] Missing object ownership check	GET /documents/{id}	Confirmed
FAPI-SEC-002	Medium	High	[Example] Over-permissive update schema	PATCH /users/{id}	Confirmed
FAPI-SEC-003	Info	Medium	[Example] Swagger UI exposed in production	/docs	Environment check

These rows are placeholders showing formatting only; they are not claims about your application.

8. Detailed Finding Template

```
## FAPI-SEC-XXX - <Specific finding title>

Severity: <Critical|High|Medium|Low|Informational>
Confidence: <High|Medium|Low>
Status: <Confirmed|Needs Environment Verification|Informational>
CWE: <CWE-ID>
OWASP API: <APIx:2023 ... or N/A>

Affected endpoints
- <METHOD /path>

Affected code
- path/to/file.py:L123-L145
```

Evidence / code path

<Trace the request-controlled input or authenticated principal through validation, authorization and the sensitive sink/resource. Explain the missing/incorrect control.>

Preconditions

<What the attacker must already have or know.>

Attack scenario

<Concise local/realistic scenario.>

Impact

<Concrete impact and scope.>

Safe reproduction / regression test

<pytest/TestClient/httpx steps using synthetic data only.>

Root cause

<Underlying design/code issue.>

Remediation

<Recommended fix.>

Minimal patch

<small diff or code sample>

Verification after fix

<Pass/Fail/Not tested + test/command reference>

9. PDF / Documentation Output Requirements

- Generate the Markdown report first. Treat it as the canonical source so PDF formatting never changes the technical finding content.
- The PDF cover should state repository, branch, commit SHA, audit date, scope and auditor.
- Include a one-page executive summary, a severity summary, and a findings summary table before detailed findings.
- Every detailed finding must include exact file/line evidence and a safe verification test or a clearly stated verification limitation.
- Add an OWASP API Top 10 coverage matrix so readers can see what was reviewed even when no vulnerability was confirmed.
- Separate confirmed vulnerabilities from hardening recommendations and environment-dependent observations.
- Redact all credentials, JWTs, cookies, API keys and PII. Never put live secret values in the PDF.
- End with remediation priorities and residual risks. Do not make a blanket claim that the backend is secure.

10. Reference Baseline

Alibaba OpenCodeReview: Repository-level AI review with deterministic rules/pipelines, LLM context and line-level comments; supports full file/directory scans. <https://github.com/alibaba/open-code-review>

OWASP API Security Top 10 - 2023: Primary API threat-model baseline used by this audit specification. <https://api-security.owasp.org/editions/2023/en/0x11-t10/>

FastAPI Security Documentation: FastAPI authentication/authorization mechanisms and security utilities. <https://fastapi.tiangolo.com/tutorial/security/>

FastAPI OAuth2/JWT: JWT and password-hashing implementation guidance. <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>

PyCQA Bandit: Python AST-based security linter for common security issues. <https://github.com/PyCQA/bandit>

PyPA pip-audit: Audits Python environments/dependencies for known vulnerabilities. <https://github.com/pypa/pip-audit>

11. Completion Checklist

- ☐ All FastAPI routes discovered and mapped.
- ☐ Authentication and authorization dependency graph reviewed.
- ☐ OWASP API Top 10 coverage completed.
- ☐ Deterministic scanner outputs captured and triaged.
- ☐ Critical/High/Medium candidates verified locally where possible.
- ☐ False positives separated from confirmed findings.
- ☐ Exact code evidence included for every confirmed vulnerability.
- ☐ Secrets redacted from artifacts.
- ☐ Remediation plan and regression tests included.
- ☐ Final Markdown/HTML/PDF generated from the same canonical findings.